

LAPORAN PRAKTIKUM INTERNET OF THINGS 2



Oleh:

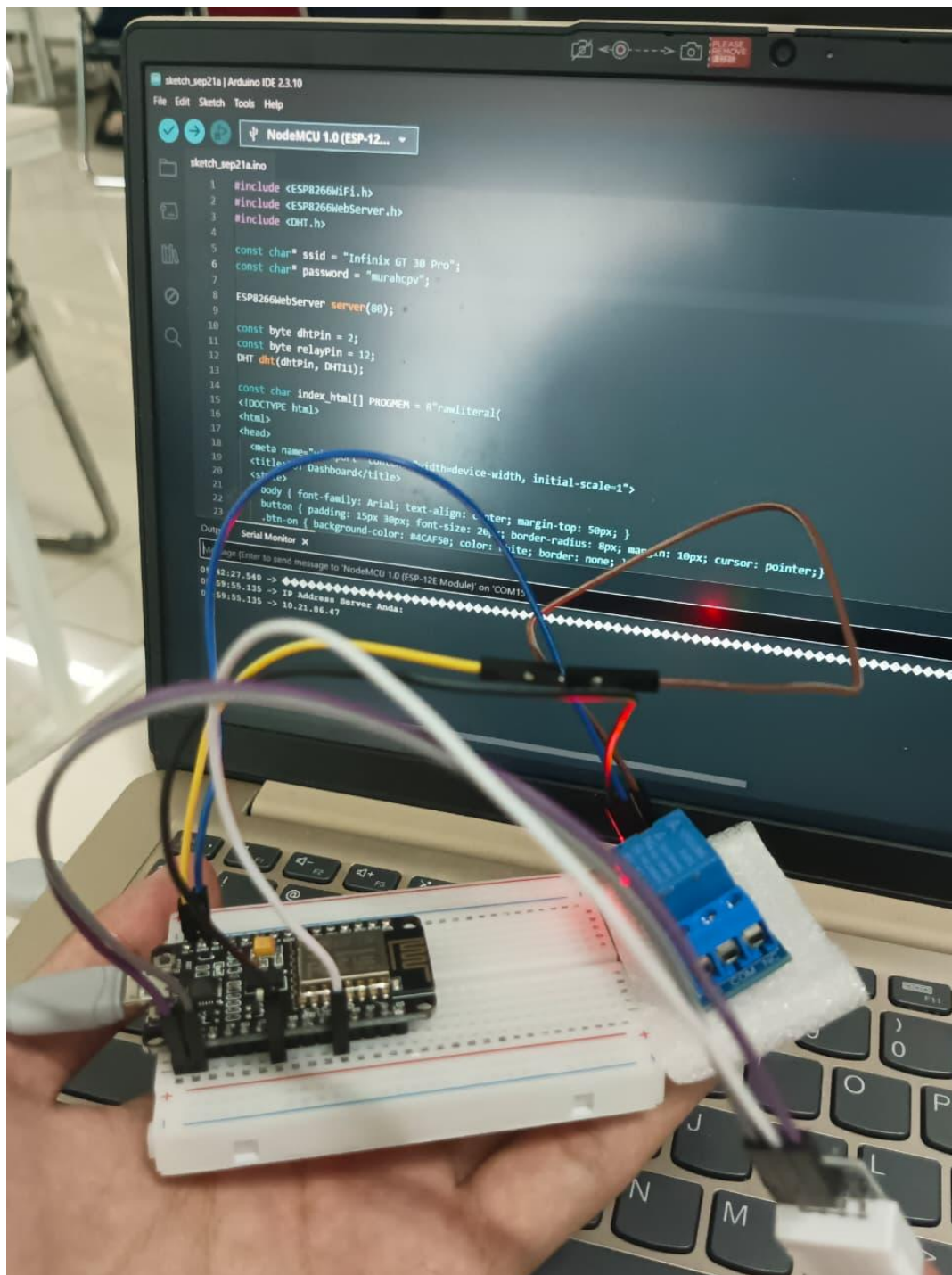
Nama : Ilham Alian Safaat
Npm : 25782012
Kelas : 3A

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2026/2027

Praktikum 1: NodeMCU sebagai *Web Server* (Menyajikan HTML)



Langkah Kerja Rangkaian:

1. Hubungkan pin **DATA / OUT / S** DHT ke pin **D4 (GPIO 2)**. (Pastikan VCC DHT ke 3V3, GND ke GND).
2. Hubungkan pin **IN / Signal** Relay ke pin **D6 (GPIO 12)**. (Pastikan VCC Relay ke VIN/VU NodeMCU, GND ke GND).

3. Untuk menguji Bersama client Relay pada server di ubah menjadi LED karena kondisi khusus

Praktikum 2: NodeMCU sebagai HTTP Client (Meminta Layanan)



Langkah Kerja Rangkaian:

1. Ubah rangkaian pertama pada bagian DHT, copot semua kabel yang terpasang dan pasang LDR
2. Rangkai sensor **LDR** dan **Resistor 10k Ohm** sebagai pembagi tegangan (seperti di Modul 2) lalu hubungkan jalur sinyalnya ke pin **A0**.

- **Latihan 1 (Pemahaman Jaringan):** Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)*?

Jawab:

Kita menggunakan mode jaringan **Station (STA)** pada kedua praktikum karena ESP8266 berperan sebagai **client** yang terhubung ke jaringan Wi-Fi yang sudah tersedia, seperti hotspot dari HP atau Access Point. Dengan mode STA, ESP8266 dapat berkomunikasi dengan perangkat lain dalam jaringan, termasuk mengirim atau menerima data dari server. Jika menggunakan mode **Access Point (AP)**, ESP8266 justru akan membuat jaringan Wi-Fi sendiri dan perangkat lain harus terhubung ke jaringan yang dibuat oleh ESP8266. Hal tersebut tidak sesuai dengan topologi praktikum yang menggunakan jaringan Wi-Fi yang sudah tersedia untuk menghubungkan **client ESP8266 dengan server**. Jadi, mode **STA lebih sesuai** karena ESP8266 membutuhkan koneksi ke jaringan yang sudah ada agar dapat melakukan komunikasi antara client dan server pada kedua praktikum.

- **Latihan 2 (Modifikasi Antarmuka & Telemetry):** Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada *Web Server*:

- i. **Menampilkan Kelembapan:** Tambahkan pembacaan kelembapan menggunakan fungsi `dht.readHumidity()`. Sisipkan placeholder `%HUMIDITY%` pada kerangka HTML di dalam tag `<div class="sensor-box">` (contoh: `<p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>`), lalu perbarui fungsi `handleRoot()` menggunakan `html.replace("%HUMIDITY%", ...)` agar nilai kelembapan muncul di halaman *web*.

- ii. **Tombol Dinamis Tunggal (*Toggle Button*):** Ubah antarmuka agar **hanya menggunakan 1 buah tombol tunggal** yang statusnya adaptif mengikuti kondisi **LED** saat ini:

- Jika LED sedang **OFF**, tombol menampilkan tulisan **"NYALAKAN (ON)"** (misal: warna hijau/biru) dan mengarahkan aksi ke `/relay/on`.
- Jika LED sedang **ON**, tombol otomatis berubah menjadi tulisan **"MATIKAN (OFF)"** (warna merah) dan mengarahkan aksi ke `/relay/off`.

(Petunjuk: Anda dapat menyematkan placeholder baru di HTML seperti

%RELAY_BUTTON%, lalu di fungsi *handleRoot()*, periksa status pin *relay/LED* menggunakan *digitalRead(relayPin)* sebelum mengganti *placeholder* tersebut dengan elemen tautan tombol yang sesuai).

Tuliskan potongan baris kode HTML dan fungsi *handleRoot()* yang Anda ubah!

```
<div class="sensor-box">

  <p>
    Suhu Saat Ini:
    <strong>%TEMPERATURE%</strong>
    Celcius
  </p>

  <p>
    Kelembapan:
    <strong>%HUMIDITY%</strong>
    %
  </p>

</div>

<h2>Kendali Relay</h2>

%RELAY_BUTTON%

</body>

</html>
)rawliteral";
```

```
if (isnan(t)) {
  html.replace("%TEMPERATURE%", "--");
} else {
  html.replace("%TEMPERATURE%", String(t));
}

// =====
// Menampilkan kelembapan
// =====

if (isnan(h)) {
  html.replace("%HUMIDITY%", "--");
} else {
  html.replace("%HUMIDITY%", String(h));
}
```

- **Latihan 3 (Analisis Kode Client):** Pada Praktikum 2, *Serial Monitor Client* mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh *Serial*

Monitor Anda jika ternyata NodeMCU *Server* teman Anda mendadak kehilangan daya (*offline*)? (Silakan cabut daya Server dan lihat hasil erornya).

Jawab:

Jika NodeMCU kawan offline maka fungsi `http.GET()` tidak menerima respons HTTP dari server dan akan menghasilkan kode error negatif, yang umumnya ditampilkan sebagai HTTP Response code: -1 pada Serial Monitor.

- **Latihan 4 (Pengembangan Algoritma):** Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi *conditional block* if di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) **HANYA JIKA** sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.

Jawab:

```
float suhu = dht.readTemperature();
```

```
if ((WiFi.status() == WL_CONNECTED) && (suhu > 35)) {
```

```
    WiFiClient client;
```

```
    HTTPClient http;
```

```
    http.begin(client, serverName);
```

```
    int httpResponseCode = http.GET();
```

```
    Serial.print("HTTP Response code: ");
```

```
    Serial.println(httpResponseCode);
```

```
    http.end();
```

```
}
```

Logika algoritmanya:

Baca suhu DHT > Suhu > 35°C? YA/TIDAK

GET Request Tidak melakukan

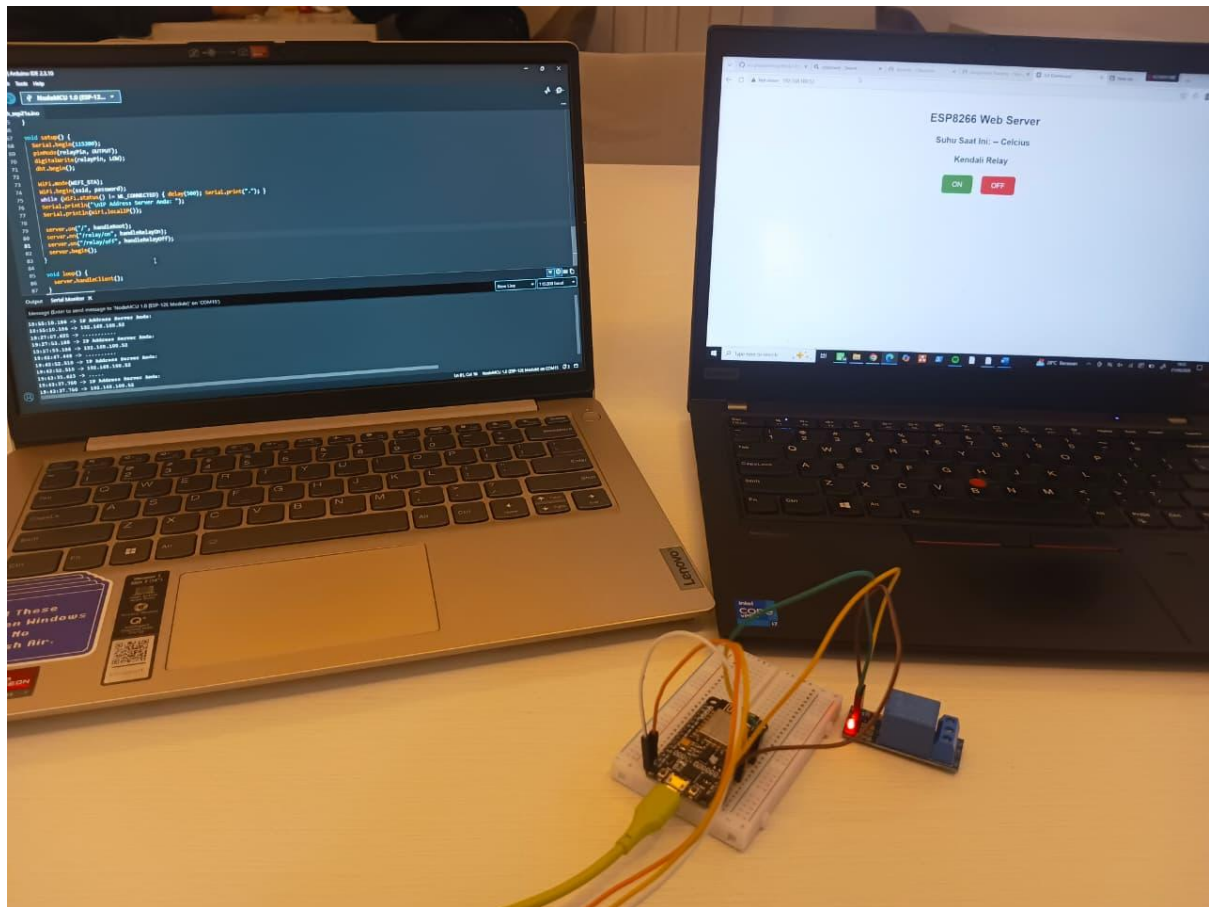
ke Server GET Request

Jadi inti conditional block yang menggantikan kondisi LDR adalah:

```
if ((WiFi.status() == WL_CONNECTED) && (suhu > 35))
```

Artinya GET Request hanya dikirim ketika Wi-Fi terhubung dan suhu melebihi 35°C.

Tugas: Pengembangan *Web Server* Cerdas dan Analisis Arsitektur



1. Langkah pengerjaan

1. Buka kode NodeMCU Server Latihan 2

Gunakan kode Server Latihan 2 yang sudah kamu buat sebelumnya, terutama bagian:

```
const char index_html[] PROGMEM = R"rawliteral(
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
...
```

```
</head>
```

2. Cari bagian `<head>`

Di dalam `<head>`, tambahkan:

```
<meta http-equiv="refresh" content="5">
```

Contohnya:

```
const char index_html[] PROGMEM = R"rawliteral(
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta http-equiv="refresh" content="5">

  <title>IoT Dashboard</title>

  <style>
    ...
  </style>
</head>
```

Pada tahap ini dilakukan modifikasi pada NodeMCU Server dari Latihan 2 untuk menambahkan fitur Auto-Refresh pada Web Dashboard. Modifikasi dilakukan pada variabel `index_html`, yaitu dengan menambahkan tag `<meta http-equiv="refresh" content="5">` di dalam bagian `<head>` HTML.

Setelah program diunggah kembali ke NodeMCU Server, halaman Web Dashboard dibuka menggunakan alamat IP Server melalui browser. Hasil pengujian menunjukkan bahwa halaman web melakukan pemuatan ulang secara otomatis setiap 5 detik tanpa perlu menekan tombol refresh pada browser. Dengan demikian, nilai suhu dan kelembapan dari sensor DHT11 dapat diperbarui secara otomatis berdasarkan kondisi terbaru sensor.

Pengujian juga dilakukan dengan menggunakan tombol kendali relay ON/OFF pada Web Dashboard. Tombol tetap dapat digunakan untuk mengendalikan relay, tetapi setiap 5 detik halaman akan dimuat ulang kembali karena adanya mekanisme Meta Refresh.